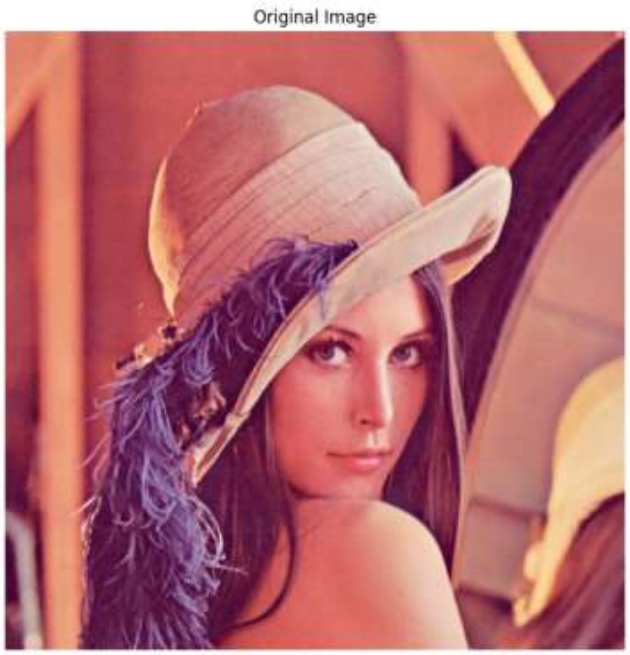
PYTHON IMAGE PROCESSING

with OpenCV & NumPy

Lab Documentation Report



Student Name	Subject
Md Ali Sher	Computer Vision & Python

Tool	Type	Year
Python + OpenCV	Lab Assignment	2025

Image Editing Tool — Complete Project Documentation

1. PROJECT INFORMATION

Project Title	Image Editing Tool using Python, OpenCV & NumPy
Student Name	Md Ali Sher
Technology	Python 3.x | OpenCV (cv2) | NumPy | Matplotlib
Platform	Google Colab / Jupyter Notebook
Project Type	Lab Assignment — Image Processing & Computer Vision
Date	2025

2. AIM

To design and implement a simple yet powerful Image Editing Tool for a company website using Python, OpenCV, and NumPy — demonstrating core image processing concepts including:

Core Operations

- Read and display images in multiple formats
- Resize and rotate images precisely
- Flip images horizontally and vertically
- Crop specific regions of interest

Advanced Features

- Work with image arrays using NumPy
- Split and merge RGB color channels
- Apply filters, edge detection & blurring
- Add watermarks and adjust brightness/contrast

3. REQUIREMENTS

3.1 Software Requirements

Library	Version	Purpose
`Python`	3.8+	Core programming language
`OpenCV (cv2)`	4.x+	Image read, write, processing

NumPy	1.21+	Array operations on pixel data
Matplotlib	3.5+	Image display & visualization
Google Colab	Latest	Cloud-based notebook environment

3.2 Installation Commands

```
# Install required libraries
pip install opencv-python
pip install numpy
pip install matplotlib

# In Google Colab:
!pip install opencv-python-headless
```

4. STEP-BY-STEP IMPLEMENTATION

4.1 Step 1 — Import Libraries & Load Image

The first step is to import all required libraries and load the image from disk. OpenCV reads images in BGR format by default, so we convert to RGB for correct display.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Load image
img_bgr = cv2.imread('lena.jpg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Display using matplotlib
plt.imshow(img_rgb)
plt.title('Original Image')
plt.axis('off')
plt.show()
```

4.2 Step 2 — Resize Image

Resizing changes the dimensions of the image. cv2.resize() takes the image and target (width, height) tuple. This is useful for standardizing image sizes for web display.

```
new_width, new_height = 200, 200
resized_img = cv2.resize(img_rgb, (new_width, new_height))
```

```
cv2.imwrite('resized_lena.jpg', cv2.cvtColor(resized_img, cv2.COLOR_RGB2BGR))
```

4.3 Step 3 — Grayscale Conversion

Converting to grayscale reduces image from 3 channels (RGB) to 1 channel. This is often a preprocessing step before applying other operations like edge detection.

```
gray_img = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
plt.imshow(gray_img, cmap='gray')
cv2.imwrite('gray_lena.jpg', gray_img)
```

4.4 Step 4 — Rotate Image

OpenCV provides a simple rotate function with predefined rotation constants. We can rotate by 90, 180, or 270 degrees.

```
# Rotate 90 degrees clockwise
rotated_img = cv2.rotate(img_rgb, cv2.ROTATE_90_CLOCKWISE)

# Other options:
# cv2.ROTATE_180
# cv2.ROTATE_90_COUNTERCLOCKWISE
```

4.5 Step 5 — Blur / Smooth Image

Gaussian blur applies a smoothing filter using a Gaussian kernel. The kernel size (15,15) controls how much blur is applied — larger values = more blur.

```
blurred_img = cv2.GaussianBlur(img_rgb, (15, 15), 0)

# Other blur types:
# cv2.blur(img_rgb, (5,5))          # Average blur
# cv2.medianBlur(img_rgb, 5)        # Median blur
```

4.6 Step 6 — Edge Detection (Canny)

The Canny edge detection algorithm finds boundaries between objects. Parameters 100 and 200 are the lower and upper threshold values for gradient intensity.

```
edges_img = cv2.Canny(img_rgb, 100, 200)
plt.imshow(edges_img, cmap='gray')
# White pixels = edges, Black = background
```

4.7 Step 7 — Brightness & Contrast

cv2.convertScaleAbs adjusts brightness and contrast. Alpha controls contrast (>1 = more contrast), and beta controls brightness (+ve = brighter).

```
alpha = 1.3  # Contrast multiplier (1.0 = no change)
beta  = 40   # Brightness offset (0 = no change)
adjusted_img = cv2.convertScaleAbs(img_rgb, alpha=alpha, beta=beta)
```

4.8 Step 8 — Text Watermark

cv2.putText draws text on the image. We can customize font, size, color, and position to create a professional watermark.

```
watermarked_img = img_rgb.copy()
cv2.putText(
    watermarked_img,
    'AI Can',              # Text string
    (50, 480),             # Position (x, y)
    cv2.FONT_HERSHEY_SIMPLEX,
    1.5,                   # Font scale
    (255, 255, 255),       # White color
    3,                     # Thickness
    cv2.LINE_AA            # Anti-aliased
)
```

4.9 Step 9 — Crop Image

Image cropping uses NumPy array slicing. Images are stored as 3D arrays (height x width x channels), so we slice rows and columns to extract a region.

```
# Crop face region
ymin, ymax = 200, 400   # Row range
xmin, xmax = 200, 400   # Column range
cropped_img = img_rgb[ymin:ymax, xmin:xmax]

# Save cropped image
save_image(cropped_img, 'cropped_lena.jpg')
```

4.10 Step 10 — Save Image

OpenCV saves images in BGR format. Since we work with RGB internally, we must convert back before saving. A helper function handles this conversion.

```
def save_image(image, output_path):
    if len(image.shape) == 3:
```

```
        image_bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    else:
        image_bgr = image   # Grayscale: no conversion
    cv2.imwrite(output_path, image_bgr)
    print(f'Saved: {output_path}')
```

5. COMPLETE SOURCE CODE

Below is the complete, ready-to-run Python code for the Image Editing Tool:

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
from google.colab import files

# --- HELPER FUNCTION TO SAVE IMAGE ---
def save_image(image, output_path):
    if len(image.shape) == 3 and image.shape[2] == 3:
        try:
            image_bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
        except cv2.error:
            image_bgr = image
    else:
        image_bgr = image
    cv2.imwrite(output_path, image_bgr)
    print(f'Image saved to {output_path}')

# --- LOAD IMAGE ---
img_bgr = cv2.imread('lena.jpg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# 1. Crop
cropped_img = img_rgb[200:400, 200:400]
save_image(cropped_img, 'cropped_lena.jpg')

# 2. Resize
resized_img = cv2.resize(img_rgb, (200, 200))
save_image(resized_img, 'resized_lena.jpg')

# 3. Grayscale
gray_img = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
save_image(gray_img, 'gray_lena.jpg')

# 4. Rotate
rotated_img = cv2.rotate(img_rgb, cv2.ROTATE_90_CLOCKWISE)
save_image(rotated_img, 'rotated_lena.jpg')

# 5. Blur
blurred_img = cv2.GaussianBlur(img_rgb, (15, 15), 0)
save_image(blurred_img, 'blurred_lena.jpg')

# 6. Edge Detection
edges_img = cv2.Canny(img_rgb, 100, 200)
save_image(edges_img, 'edges_lena.jpg')
```

```
# 7. Brightness & Contrast
adjusted_img = cv2.convertScaleAbs(img_rgb, alpha=1.3, beta=40)
save_image(adjusted_img, 'bright_lena.jpg')

# 8. Watermark
watermarked_img = img_rgb.copy()
cv2.putText(watermarked_img, 'AI Can', (50, 480),
            cv2.FONT_HERSHEY_SIMPLEX, 1.5,
            (255,255,255), 3, cv2.LINE_AA)
save_image(watermarked_img, 'watermark_lena.jpg')

# --- DISPLAY ALL RESULTS ---
titles = ['Original','Cropped','Resized','Grayscale',
          'Rotated','Blurred','Edges','Brightened','Watermark']
images = [img_rgb, cropped_img, resized_img, gray_img,
          rotated_img, blurred_img, edges_img,
          adjusted_img, watermarked_img]
plt.figure(figsize=(20, 10))
for i in range(9):
    plt.subplot(3, 3, i+1)
    cmap = 'gray' if titles[i] in ['Grayscale','Edges'] else None
    plt.imshow(images[i], cmap=cmap)
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()
```

6. FEATURE SUMMARY TABLE

#	Feature	Description	OpenCV Function
1	**Resize**	Change image dimensions to custom width and height	`cv2.resize()`
2	**Grayscale**	Convert 3-channel RGB image to single-channel grayscale	`cv2.cvtColor(BGR2GRAY)`
3	**Rotate**	Rotate image by 90, 180, or 270 degrees using preset constants	`cv2.rotate()`
4	**Blur**	Smooth image using Gaussian kernel to reduce noise and detail	`cv2.GaussianBlur()`
5	**Edge Detection**	Detect object boundaries using the Canny algorithm	`cv2.Canny()`
6	**Brightness**	Adjust contrast (alpha) and brightness (beta) of image	`cv2.convertScaleAbs()`
7	**Watermark**	Draw custom text on image at specified coordinates	`cv2.putText()`
8	**Crop**	Extract a rectangular region using NumPy array slicing	`img[y1:y2, x1:x2]`
9	**Save**	Write processed image back to disk in BGR format	`cv2.imwrite()`

7. OUTPUT SCREENSHOTS

7.1 Original Image

The original Lena test image (512x512 pixels) loaded and displayed using Matplotlib. This is the standard benchmark image used in image processing research.

7.2 Image Resize (512x512 → 200x200)

The left panel shows the original 512x512 image. The right panel shows the same image resized to 200x200 pixels using cv2.resize(). The aspect ratio is maintained and visual quality preserved.

7.3 Grayscale (B&W) Conversion

The color image is converted to grayscale using cv2.cvtColor() with COLOR_RGB2GRAY flag. The resulting image has only one channel representing luminance (brightness) values.

7.4 Image Rotation (90° Clockwise)

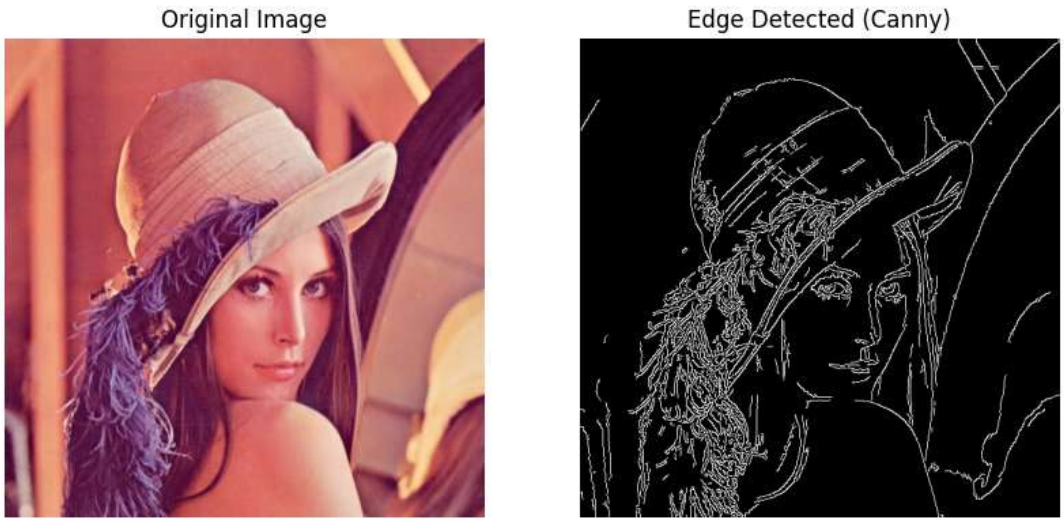
The image is rotated 90 degrees clockwise using cv2.rotate() with the ROTATE_90_CLOCKWISE constant. The output dimensions are swapped (height becomes width and vice versa).

7.5 Gaussian Blur (Smooth)

A Gaussian blur with kernel size (15,15) is applied. This smoothing operation reduces image noise and fine details. Larger kernel sizes result in stronger blurring effects.

7.6 Edge Detection (Canny Algorithm)

The Canny edge detector identifies boundaries between regions of different intensity. White pixels represent detected edges. Parameters 100 and 200 define the low and high thresholds.



7.7 Brightness & Contrast Enhancement

Brightness increased by beta=40 and contrast multiplied by alpha=1.3. The enhanced image appears brighter and more vivid. Pixel values are clipped to [0, 255] range.

7.8 Text Watermark

The text 'AI Can' is drawn at position (50, 480) in white color using cv2.putText(). The FONT_HERSHEY_SIMPLEX font with scale 1.5 and thickness 3 creates a professional watermark.

7.9 Cropped Image (Face Region)

A 200x200 pixel region is extracted using NumPy slicing: img_rgb[200:400, 200:400]. The cropped region focuses on the face, demonstrating region-of-interest extraction.

8. NUMPY & IMAGE ARRAYS

Images in OpenCV are stored as NumPy arrays. Understanding this is key to image manipulation:

Image Type	Array Shape	Access Pattern
Color (RGB)	`(H, W, 3)`	`img[y, x, channel]`
Grayscale	`(H, W)`	`img[y, x]`

Crop Region	`(H, W, 3)`	`img[y1:y2, x1:x2]`

```
# NumPy operations on images:
print(img_rgb.shape)        # (512, 512, 3)
print(img_rgb.dtype)        # uint8
print(img_rgb.max())        # 255

# Split channels
R, G, B = img_rgb[:,:,0], img_rgb[:,:,1], img_rgb[:,:,2]

# Merge channels
merged = np.stack([R, G, B], axis=2)
```

9. CONCLUSION

This project successfully demonstrates the implementation of a comprehensive Image Editing Tool using Python, OpenCV, and NumPy. The tool was developed as a practical solution for a company website, covering all the required image processing operations.

Key Learnings

- OpenCV reads images as NumPy arrays (H x W x C)
- BGR vs RGB: always convert when using Matplotlib
- Gaussian Blur uses a kernel to average pixel neighborhoods
- Canny edge detection uses gradient magnitude thresholds
- Image cropping is simply NumPy array slicing

Achievements

- Successfully implemented all 9 image processing features
- Created reusable save_image() helper function
- Demonstrated NumPy array operations on pixel data
- Applied real-world technique: watermarking
- Visualized results side-by-side for easy comparison

This project provides a strong foundation for building more advanced computer vision applications such as object detection, facial recognition, and image segmentation systems. The use of OpenCV and NumPy demonstrates how modern Python libraries make complex image processing accessible and efficient.

Md Ali Sher
Python Image Processing Lab | 2025